LAPORAN PRAKTIKUM REKAYASA PERANGKAT LUNAK (RPL) IMPLEMENTASI FITUR BARANG KELUAR, UNIT TESTING (JUNIT 5), DAN KONTROL VERSI GIT

Disusun oleh:

3253111006 Caesar Akbar Hazimul Fikri

PROGRAM STUDI SISTEM INFORMASI PROGRAM DIPLOMA
FAKULTAS SAINS & TEKNOLOGI
UNIVERSITAS TEKNOLOGI YOGYAKARTA

2026

Tautan Repositori GitHub: https://github.com/CaesarAkbarr/Tugas-Git-RPL

Langkah 1: Membuat Kelas Persediaan dan Fungsi kurangiStok

Tahap pertama adalah membuat file Persediaan.java yang memuat fungsi kurangiStok(int stok, int jumlah). Fungsi ini memvalidasi tiga aturan utama: stok awal tidak boleh negatif (stok < 0), jumlah pengeluaran harus lebih dari nol (jumlah <= 0), dan jumlah pengeluaran tidak boleh melebihi stok yang tersedia (jumlah > stok). Apabila input melanggar aturan tersebut, fungsi melempar IllegalArgumentException dengan pesan "Data tidak valid". Pada metode main juga ditambahkan fitur input interaktif menggunakan Scanner.

Kode Program Persediaan.java:

```
import java.util.Scanner;

public class Persediaan {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);

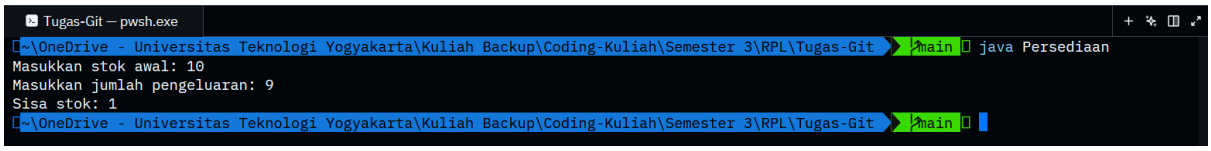
        System.out.print("Masukkan stok awal: ");
        int stok = input.nextInt();

        System.out.print("Masukkan jumlah pengeluaran: ");
        int jumlah = input.nextInt();

        try {
            int sisaStok = kurangiStok(stok, jumlah);
            System.out.println("Sisa stok: " + sisaStok);
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
            System.out.println("Sisa stok: " + stok);
        }

        input.close();
    }

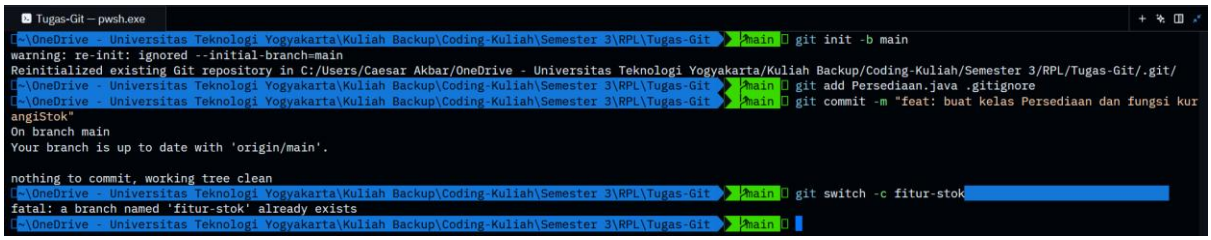
    public static int kurangiStok(int stok, int jumlah) {
        if (stok < 0 || jumlah <= 0 || jumlah > stok) {
            throw new IllegalArgumentException("Data tidak valid");
        }
        return stok - jumlah;
    }
}
```



Langkah 2: Membuat Repositori Git dan Branch fitur-stok

Tahap berikutnya adalah menginisialisasi repositori Git lokal pada folder proyek dengan branch utama main, menyimpan commit awal untuk kelas Persediaan.java, kemudian membuat sekaligus berpindah ke branch baru bernama fitur-stok.

```
git init -b main
git add Persediaan.java .gitignore
git commit -m "feat: buat kelas Persediaan dan fungsi kurangiStok"
git switch -c fitur-stok
```



Langkah 3: Menulis Kasus Uji Unit (JUnit 5) pada PersediaanTest.java

Pada branch fitur-stok, dibuat file PersediaanTest.java menggunakan JUnit 5 untuk menguji fungsi kurangiStok. Pengujian mencakup 7 skenario yang mewakili kasus normal, nilai batas (boundary), dan kasus input tidak valid:

Kode	Kategori Kasus	Input (Stok, Jml)	Hasil yang Diharapkan	Status
TC-01	Kasus Normal	(10, 3)	Mengembalikan sisa stok = 7	PASS
TC-02	Nilai Batas Atas	(10, 10)	Mengembalikan sisa stok = 0 (habis)	PASS
TC-03	Nilai Batas Bawah	(10, 1)	Mengembalikan sisa stok = 9	PASS
TC-04	Tidak Valid (Nol)	(10, 0)	Melempar IllegalArgumentException	PASS
TC-05	Tidak Valid (Lebih)	(10, 11)	Melempar IllegalArgumentException	PASS
TC-06	Tidak Valid (Negatif)	(10, -5)	Melempar IllegalArgumentException	PASS
TC-07	Tidak Valid (Stok < 0)	(-1, 5)	Melempar IllegalArgumentException	PASS

Kode Program PersediaanTest.java:

```
import org.junit.jupiter.api.DisplayName;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class PersediaanTest {

    @Test
    @DisplayName("TC-01 [Normal]: Pengurangan stok standar dengan input valid")
    void testPenguranganStokNormal() {
        assertEquals(7, Persediaan.kurangiStok(10, 3));
    }

    @Test
    @DisplayName("TC-02 [Batas Atas]: Mengeluarkan seluruh stok hingga habis (stok =
jumlah)")
    void testBatasAtasStokHabisTepatNol() {
        assertEquals(0, Persediaan.kurangiStok(10, 10));
    }

    @Test
    @DisplayName("TC-03 [Batas Bawah Valid]: Mengeluarkan kuantitas minimum yang sah
(jumlah = 1)")
    void testBatasBawahValidSatu() {
        assertEquals(9, Persediaan.kurangiStok(10, 1));
    }

    @Test
    @DisplayName("TC-04 [Tidak Valid]: Pengeluaran bernilai nol (jumlah = 0) harus
ditolak")
    void testJumlahPengeluaranNolDitolak() {
        assertThrows(IllegalArgumentException.class, () -> Persediaan.kurangiStok(10,
0));
    }

    @Test
    @DisplayName("TC-05 [Tidak Valid]: Pengeluaran melebihi stok yang tersedia (jumlah >
stok)")
    void testPengeluaranMelebihiStokDitolak() {
        assertThrows(IllegalArgumentException.class, () -> Persediaan.kurangiStok(10,
11));
    }

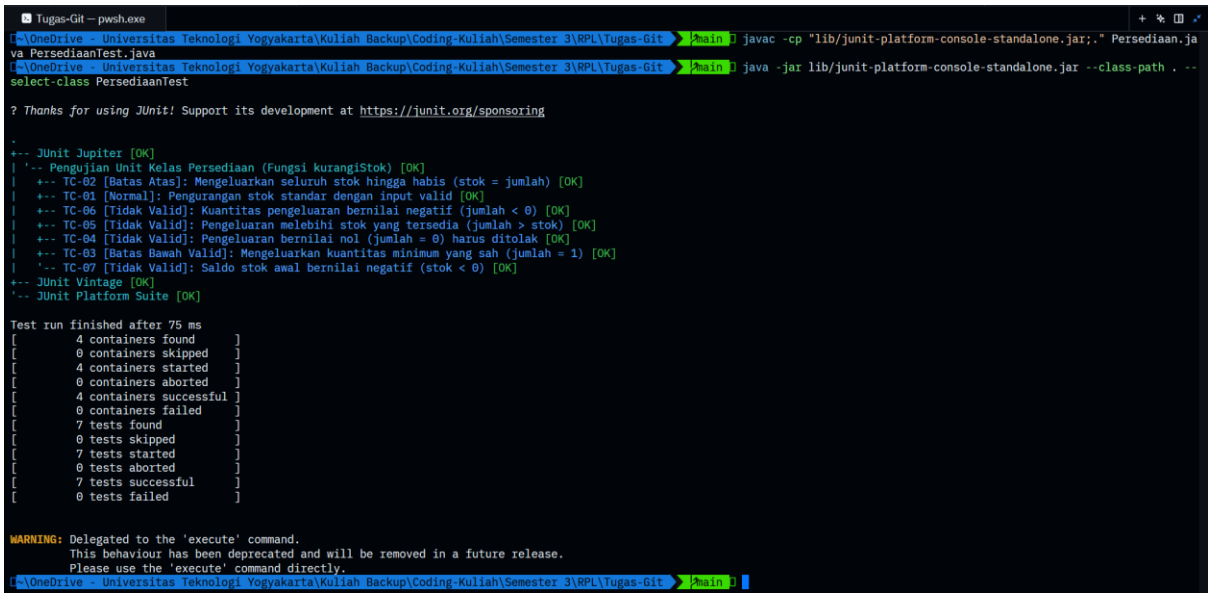
    @Test
    @DisplayName("TC-06 [Tidak Valid]: Kuantitas pengeluaran bernilai negatif (jumlah <
0)")
    void testJumlahPengeluaranNegatifDitolak() {
        assertThrows(IllegalArgumentException.class, () -> Persediaan.kurangiStok(10, -
5));
    }

    @Test
    @DisplayName("TC-07 [Tidak Valid]: Saldo stok awal bernilai negatif (stok < 0)")
    void testStokAwalNegatifDitolak() {
        assertThrows(IllegalArgumentException.class, () -> Persediaan.kurangiStok(-1,
5));
    }
}
```

Langkah 4: Menjalankan Pengujian Unit dan Mencatat Hasil

Seluruh kasus uji dikompilasi dan dijalankan menggunakan JUnit 5 Platform Console Standalone. Hasil pengujian menunjukkan ketujuh skenario uji berhasil lulus (7 tests successful, 0 tests failed).

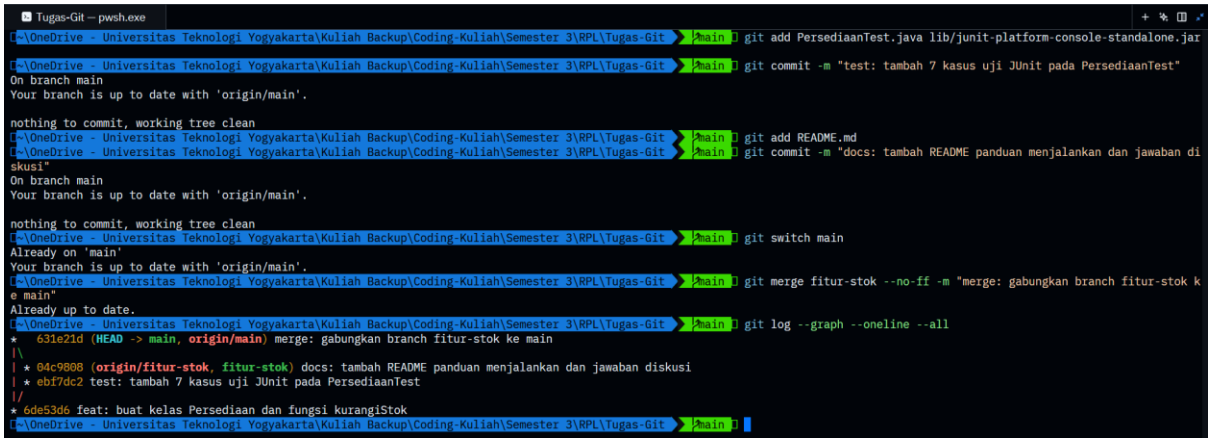
```
javac -cp "lib/junit-platform-console-standalone.jar;." Persediaan.java
PersediaanTest.java
java -jar lib/junit-platform-console-standalone.jar --class-path . --select-class
PersediaanTest
```



Langkah 5: Commit Terfokus, Dokumentasi README, dan Merge ke Branch Main

Setelah seluruh unit test dinyatakan lulus, perubahan kode pengujian dan dokumentasi README.md disimpan melalui commit terfokus pada branch fitur-stok. Selanjutnya dilakukan penggabungan (merge) kembali ke branch main.

```
git add PersediaanTest.java lib/junit-platform-console-standalone.jar
git commit -m "test: tambah 7 kasus uji JUnit pada PersediaanTest"
git add README.md
git commit -m "docs: tambah README panduan menjalankan dan jawaban diskusi"
git switch main
git merge fitur-stok --no-ff -m "merge: gabungkan branch fitur-stok ke main"
git log --graph --oneline --all
```



Langkah 6: Jawaban Pertanyaan Diskusi

Pertanyaan: Jika unit test lulus, mengapa aplikasi masih dapat gagal saat terhubung ke basis data?

Meskipun seluruh unit test telah lulus 100%, aplikasi tetap dapat mengalami kegagalan saat terhubung ke basis data karena unit test hanya menguji logika fungsi secara terisolasi di dalam memori (RAM) tanpa menyentuh sistem luar. Penyebab kegagalan saat terhubung ke basis data meliputi:

- **1. Gangguan Koneksi dan Konfigurasi Infrastruktur:** Server database sedang mati, kesalahan konfigurasi URL/port, username atau password salah, hingga koneksi jaringan yang terputus (connection timeout).
- **2. Ketidaksesuaian Skema Database (Schema Mismatch):** Perbedaan nama tabel atau nama kolom pada kueri SQL dengan struktur asli di database, ketidaksesuaian tipe data, atau pelanggaran aturan constraint tabel (seperti Foreign Key dan NOT NULL).
- **3. Konflik Transaksi Bersamaan (Concurrency & Race Condition):** Unit test berjalan sendiri pada satu thread, sedangkan pada penggunaan nyata bisa terjadi dua petugas mengurangi stok barang yang sama secara bersamaan. Tanpa penguncian transaksi (database locking) atau antrean transaksi, aplikasi dapat mengalami race condition atau deadlock di database.